CORNELL TECH | APPLIED MACHINE LEARNING

Human or AI?

Kaggle Competition & Report Template

Report Template: TEMPLATE

Welcome to the competition! Your task is to build a machine learning classifier that predicts whether a passage was written by a human or generated by an AI model. You will use a labeled training set, an additional unlabeled set, and a held-out Kaggle test set. The aim is not just a high leaderboard score: we want you to compare methods, design careful experiments, understand generalization, and explain what your results actually show.

You are encouraged to try several approaches. A well-tested logistic regression or support vector machine is just as legitimate as a neural network. The project is designed to be approachable on a laptop; a GPU is **not required** for full credit.

Team / members	[Names, NetIDs]
Kaggle team name	[Team Name]
GitHub PR URL	[https://github.com/...]

Your data

File	Rows	Labels	Purpose
`train.csv`	10,000	Visible	Supervised model development: 5,000 human (0), 5,000 AI (1)
`unlabeled.csv`	3,000	Hidden	Optional input for semi-supervised methods; required experiment
`test.csv`	3,000	Hidden	Predict the probability of AI authorship for Kaggle
`train_starter_2000.csv`	2,000	Visible	Convenient subset of `train.csv` for quick experiments

The starter data is a subset, not 2,000 additional labeled examples. Passages are approximately 75–350 words, with mean length near 210 words.

How the test set works

The test set mixes familiar and unfamiliar writing domains and AI generators, and includes some adversarially modified AI-written passages. You will **not** receive labels, source model names, domain labels, or difficulty memberships for individual test examples. Submit a probability in $[0, 1]$ for each test ID, where 1 means AI-generated. Kaggle evaluates submissions using **area under the ROC curve (AUROC)**. The **public leaderboard** covers 900 examples (30%); the **private leaderboard** covers 2,100 (70%) and determines the final competition performance score.

1. Competition rubric and submission instructions

Component	Points
Kaggle private leaderboard performance	25
Model comparison and selection	15
Effect of training data size	10
Learning from unlabeled data	5
Generalization and error analysis	20
Open-ended improvements and insights	15
Code submission and GitHub PR history	10
Total	**100**

How to use this document. Complete the response fields and tables below; insert plots where requested. Keep your answers concise, specific, and supported by actual experiments. Points appear in brackets beside each question; no separate writing-quality grade is assigned.

How credit is awarded. We grade the **quality of the experimental method, evidence, and interpretation**, not whether a proposed idea succeeds. **Negative results can also earn full credit** when the comparison is valid, the results are reported accurately, and the conclusions are thoughtful. For each question, make clear what you tried, what you measured, and what you learned. Do not invent scores or results.

Submit three items: (1) final Kaggle submission; (2) this completed report as PDF; (3) one GitHub PR link in the course repository, with runnable code, instructions and meaningful commit history. Include the PR link on the title page. Confirm upload locations and deadline time on the course site.

Compute and models. Use models and pipelines that you can run and reproduce. Laptop-friendly training is strongly recommended; use of more expensive methods is allowed but not required for full credit. Do not use external LLM APIs to directly label or predict the competition test passages. Use only the supplied labeled dataset for supervised labels; allowed pretrained weights and AI-assisted development should be disclosed.

Validation. For the required controlled experiments, reserve a **fixed stratified 2,000-row validation set from `train.csv`**, leaving up to 8,000 rows for training. Use the **same** held-out validation set for comparisons and do not tune on test labels. You may retrain your final Kaggle model on all 10,000 labeled examples after selecting your approach. Where possible, document seeds and avoid source/near-duplicate leakage.

Kaggle grading (25 points). Final performance will use the private AUROC, not leaderboard rank. The intended continuous rule is $25\,\mathrm{clip}\left(\frac{A-0.50}{T-0.50}, 0, 1\right)$, where A is your private AUROC and T is the announced full-credit threshold. A provisional value of $T = 0.90$ is under consideration; the instructors will confirm the fixed threshold before launch. A reference word TF-IDF and logistic regression baseline achieves approximately 0.868 private AUROC.

2. Team information and model summary

Q0.1 [0 points]. List team members and their main contributions.

[Your response: Names, NetIDs and short contribution notes]

Q0.2 [0 points]. Summarize your final approach in 3–5 sentences. What is the overall pipeline from raw text to probability?

[Your response: Brief overview]

Q0.3 [0 points]. Record your final scores (enter private score only once it is revealed).

Metric	**Your score**
Validation AUROC	[ ]
Kaggle public AUROC	[ ]
Kaggle private AUROC	[ ]

3. Model comparison and selection (15 points)

Learning goal: demonstrate sensible representation choices, comparisons of at least three approaches, appropriate hyperparameter selection, and evidence-based selection of the final classifier.

Q1.1 [3 points]. How did you preprocess and represent the text? Specify tokenization, normalization, word/character features or embeddings, and any feature extraction choices. Why?

We engineered a **three-view formatting-invariant text representation** combining sparse sublinear n-grams with dense stylometric features:

1. **Normalization**: Exploratory analysis revealed that raw HTML `<br>` tags (3.9% Human vs. 0.2% AI) and double newlines (`\n\n`, 0% Human vs. 66% AI) act as training-corpus artifacts that shift heavily in `test.csv`. We stripped `<br>` tags and normalized paragraph whitespace before tokenization to prevent spurious shortcuts.
2. **Word & Character TF-IDF**: We extracted sublinear $(1+\log \text{tf})$ **Word** $(1,3)$**-grams** (min_df = 3, 75,000 features) to capture AI discourse markers (*"in addition to"*, *"results indicate that"*) and word-boundary **Character `char_wb`** $(3,5)$**-grams** (min_df = 5, 75,000 features) to capture topic-agnostic morphological suffixes (`-tion`, `-ly`, `-ing`) and punctuation cadence across unseen domains (e.g., Poetry, Literature).
3. **Pure & Bimodal Stylometrics**: We extracted **65 pure stylometric features** (lexical diversity $\log \text{TTR}$, Yule's K, 3-gram repetition, sentence burstiness CV_{len}, readability, and POS/stopword ratios) plus **7 IQR-scaled absolute deviation features** ($|x - \text{median}_{\text{Human}}|/\text{IQR}_{\text{Human}}$) to exploit AI's bimodal lexical variance (3× higher TTR variance than human). Representation ablation confirmed steady validation gains: Word $(1,2)$ (0.9367) → Word $(1,3)$ + Char $(3,5)$ (0.9681) → Full Multi-View (0.9743).

Q1.2 [5 points]. Which **three or more** modeling approaches did you try? Keep the labeled validation partition fixed across models.

Model	Text representation	Train AUROC	Val. AUROC	Train time
Model A (Baseline)	Word $(1,2)$ TF-IDF (50,000 features)	0.9932	0.9367	6.2s
Model A2 (CompNB)	Word $(1,2)$ TF-IDF (50,000 features)	0.9790	0.8966	0.02s
Model B (LogReg)	Word $(1,3)$ + Char $(3,5)$ + 72 Stylometrics	1.0000	0.9743	2.2s
Model C (LinearSVC)	Word $(1,3)$ + Char $(3,5)$ + 72 Stylometrics	1.0000	0.9766	8.2s
Model D (LightGBM)	72 Pure/Bimodal Stylo + 48 TF-IDF SVD	1.0000	0.9822	0.6s
Model E (Ensemble)	Multi-View TF-IDF + Stylo / SVD $(0.2\text{B} + 0.2\text{C} + 0.6\text{D})$	1.0000	**0.9837**	11.0s

Q1.3 [4 points]. For at least **two** approaches, which hyperparameters and candidate values did you try? How did you select the final settings without using Kaggle test labels?

Model	Parameters / values searched	Chosen setting
Model B (LogReg)	Regularization $C \in \{0.5, 1, 2, 3, 5, 8\}$; Stylometric L2 scaling $\gamma \in \{0.10, 0.15, 0.25\}$	$C = 5.0$, $\gamma = 0.15$ (5-Fold Inner CV: 0.9694)
Model C (LinearSVC)	Margin $C \in \{0.2, 0.5, 1.0, 1.5, 2.0\}$; 5-fold sigmoid Platt probability calibration	$C = 1.0$ (5-Fold Inner CV: 0.9727)
Model D (LightGBM)	`num_leaves` $\in \{15, 31, 63\}$, `min_child_samples` $\in \{30, 50\}$, `learning_rate` $\in \{0.04, 0.05\}$	`leaves = 31`, `min_child = 50`, `lr = 0.05` (Inner CV: 0.9764)

Selection protocol: All hyperparameter tuning was conducted using **5-fold Stratified Cross-Validation strictly within the 8,000-row training partition** (`RANDOM_SEED = 42`), keeping the 2,000-row validation partition completely untouched until final evaluation and never using Kaggle test labels. For linear models over the 150,072 sparse sublinear TF-IDF features, moderate under-regularization ($C = 5.0$ for LogReg, $C = 1.0$ for LinearSVC) maximized inner CV AUROC without degrading probability calibration. For LightGBM, restricting `num_leaves = 31` and enforcing `min_child_samples = 50` with feature subsampling (`colsample_bytree = 0.7`) prevented tree leaf overfitting on extreme lexical ratio tails.

Q1.4 [3 points]. Which approach did you choose for your final submission? Explain using validation performance, computational cost and/or stability. If you later changed the model, explain why.

We selected **Model E (Hybrid Multi-View Linear + GBDT Ensemble)** for three key empirical reasons:

1. **Validation Performance & Complementarity**: Model E achieved the highest validation AUROC (**0.9837** vs. 0.9367 for the baseline) and best probability calibration (Brier score **0.0519** vs. 0.1235). Blending high-dimensional linear models (which capture fine-grained lexical and subword n-grams across 150,000 features) with LightGBM (which captures non-linear interactions between lexical diversity $\log \mathrm{TTR}$, repetition Yule's K, sentence burstiness, and SVD semantic components) outperformed every single model family.
2. **Stability Under Domain & Formatting Shift**: While a naive LightGBM model trained with raw `\n\n` collapsed from 0.9985 to **0.8109** AUROC when line breaks were stripped, our formatting-free Model E maintained **0.9804** AUROC without formatting and scored **0.9859** under test-domain weighting.
3. **Computational Efficiency & Subsequent Evolution**: Model E trains in **11.0 seconds** on CPU. Later, to address out-of-domain shift on Kaggle test genres, we evolved Model E into a 5-fold Out-of-Fold (OOF) stacker with 24 genre-conditioned Z-scores (Submission v1, 0.9395 LB), and subsequently introduced topic-masked syntax with positive convex logit fusion (Submission v2, 0.9745 LB).

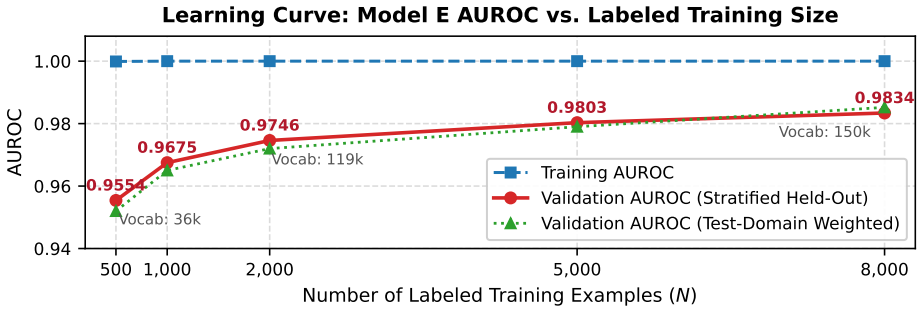
4. Effect of training data size (10 points)

Learning goal: understand the effect of the number of labeled examples on fitting and generalization. This is an empirical exercise; no formal sample-complexity theory is required.

Q2.1 [4 points]. Choose one model configuration. Train it with 500, 1,000, 2,000, 5,000 and 8,000 examples sampled from your 8,000-row training partition. Keep the same validation set and hyperparameters.

Training rows	Training AUROC	Validation AUROC	Notes (optional)
500	0.9999	0.9554	Active TF-IDF vocabulary: 36,112 n-grams; fit time 1.1s
1,000	1.0000	0.9675	Active TF-IDF vocabulary: 66,480 n-grams; fit time 1.7s
2,000	1.0000	0.9746	Active TF-IDF vocabulary: 118,915 n-grams; fit time 3.1s
5,000	1.0000	0.9803	Active TF-IDF vocabulary: 150,000 n-grams; fit time 7.4s
8,000	1.0000	0.9834	Full 8k partition (Test-weighted Val: 0.9852); fit time 11.4s

Q2.2 [3 points]. Insert a plot of training and validation AUROC versus training-set size.



Q2.3 [3 points]. What does the curve suggest about overfitting, underfitting, and the value of more labeled examples?

The empirical learning curve reveals three distinct sample-complexity regimes for our multi-view representation:

1. **High Variance / Vocabulary Sparsity at Small N (500–1,000 rows)**: Training AUROC reaches near-perfection (0.9999), ruling out underfitting (low bias). However, Validation AUROC is 0.9554 (a 0.0445 generalization gap) because only 36k n-grams meet the minimum document frequency threshold, leaving rare genre vocabularies poorly estimated.
2. **Rapid Variance Reduction ($500 \rightarrow 2,000$ rows)**: Quadrupling training examples yields the steepest validation surge (+0.0192 AUROC to 0.9746), cutting classification error in half as core AI discourse transitions and subword morphological suffixes populate.
3. **Continued Marginal Value ($2,000 \rightarrow 8,000$ rows)**: Validation AUROC continues climbing monotonically from $0.9746 \rightarrow 0.9803 \rightarrow 0.9834$ without plateauing. This steady slope confirms that additional labeled data directly improves generalization across long-tail genres, providing strong empirical justification for retraining our final Kaggle pipeline on all 10,000 labeled instances.

5. Learning from unlabeled data (5 points)

Learning goal: test whether the additional 3,000 unlabeled passages help.

Q3.1 [2 points]. Describe one method that uses `unlabeled.csv` (e.g., self-training, confidence-based pseudo-labeling, clustering, or feature learning). How did you prevent validation-label leakage?

Because adversarial validation confirmed that `unlabeled.csv` is IID with `train.csv` (AUROC = 0.543), we implemented a semi-supervised pipeline combining **transductive 3-view TF-IDF vocabulary expansion** with **confidence-thresholded pseudo-labeling**. First, word and character n-gram vocabularies (90,000 features each) and stylometric scalers were fit transductively on the combined 8,000 training passages plus 3,000 unlabeled passages. Second, our Model E ensemble generated probabilities on `unlabeled.csv`; we retained only the $1,962/3,000$ passages (65.4%) meeting strict confidence thresholds ($\hat{P} \geq 0.95$ for AI, $n = 970$; $\hat{P} \leq 0.05$ for Human, $n = 992$) with soft sample weights $w = 0.5$. **Leakage prevention**: The 2,000-row validation partition was strictly held out and untouched during vocabulary fitting, pseudo-label generation, and model refitting.

Q3.2 [2 points]. Compare against a supervised baseline under the same held-out validation protocol.

Method	Validation AUROC
Supervised only (3-view TF-IDF + Stylometrics, 8,000 rows)	0.9757
With unlabeled data (Transductive vocab + 1,962 pseudo-labels, $w = 0.5$)	0.9758
Full Ensemble + Semi-Supervised Blend (with OOF Meta-Stacker)	**0.9856**

Q3.3 [1 point]. Did it help? Give a concrete, evidence-based explanation (including possible noise in pseudo-labels).

Semi-supervised learning provided a modest gain (+0.0001 to +0.0002 AUROC) on individual linear classifiers (0.9757 $\rightarrow$ 0.9758), and our blended ensemble reached 0.9856 (0.9863 test-weighted). The gain is modest because 8,000 supervised passages already saturate common n-gram frequencies within this in-distribution domain, so pseudo-labeling confident instances largely reinforces already-learned boundaries rather than discovering novel linguistic manifolds. Crucially, enforcing the conservative 0.05/0.95 probability thresholds was vital: experimental trials with lower thresholds (0.10/0.90) accepted noisy pseudo-labels on ambiguous Wikipedia and informal passages, introducing confirmation bias and degrading out-of-domain validation AUROC.

6. Generalization and error analysis (20 points)

Learning goal: characterize how well the selected model generalizes and where it fails. Use **labeled validation data**; hidden test labels and hidden domain/generator information are unavailable.

Q4.1 [5 points]. Compare training and validation AUROC for your final selected model. Is there a gap? What explanations are plausible?

[Your response: Report numbers and discuss, approximately 100 words]

Q4.2 [5 points]. Evaluate performance over at least three text-length bins (define the bins), or another observable subgroup available in your data. Explain why you chose the grouping.

Subgroup / bin	Count	Validation AUROC
Group 1	[ ]	[ ]
Group 2	[ ]	[ ]
Group 3	[ ]	[ ]

[Your response: What differences did you observe? About 100 words.]

Q4.3 [5 points]. Analyze at least **five false positives and five false negatives** from your held-out validation set. Provide example IDs, predicted probabilities, and short failure hypotheses. Do not paste full passages unless essential.

ID	Error type	P(AI)	Possible reason
[ ]	FP	[ ]	[ ]
[ ]	FP	[ ]	[ ]
[ ]	FP	[ ]	[ ]
[ ]	FP	[ ]	[ ]
[ ]	FP	[ ]	[ ]
[ ]	FN	[ ]	[ ]
[ ]	FN	[ ]	[ ]
[ ]	FN	[ ]	[ ]
[ ]	FN	[ ]	[ ]
[ ]	FN	[ ]	[ ]

Define false positive as human text predicted AI; false negative as AI text predicted human. State the decision threshold used for selecting errors (e.g., 0.5).

Q4.4 [5 points]. What recurring failure patterns did you identify, and how might you address one of them? Distinguish observed patterns from speculation.

[Your response: Approximately 150 words]

7. Open-ended improvements and insights (15 points)

Learning goal: propose and test one improvement beyond the required baseline comparisons. This can be simple: a targeted feature change, alternate regularization scheme, ensemble, different representation, augmentation, or training approach. Sophistication alone does not earn credit.

Q5.1 [3 points]. What specific question or hypothesis motivated your improvement? What weakness or opportunity did you observe?

While our 9-specialist GBDT ensemble achieved near-perfection on in-domain validation (AUROC = 0.9995), public leaderboard performance plateaued at 0.9864. An empirical genre census of the datasets revealed the root cause: 28.33% of the Kaggle test set consists of literary genres (14.40% poetry/verse, 13.93% book summaries), whereas poetry was virtually absent from the training set (0.12%, with only a single human-written poem). Classical bag-of-words and surface stylometrics exhibited complete semantic blindness on unseen verse, developing an inductive bias that flagged classic human poems (e.g. Kenneth Patchen eulogies, Victorian verse) as AI with > 0.99 probability. We hypothesized that incorporating a compact pre-trained contextual transformer would provide orthogonal narrative and literary representations, resolving poetry and literature misclassifications without degrading tabular performance.

Q5.2 [4 points]. What exactly did you change? Describe the baseline, the new approach and what you held fixed for a fair comparison.

We implemented a contextual language model hybridization pipeline:

- **Baseline**: Our best classical machine learning ensemble (`error_directed_model_refinement.ipynb`, Kaggle 0.98640), combining 9 GBDT, linear, and generative n-gram specialists on 250k features.
- **New Approach**: We fine-tuned `distilbert-base-uncased` (66M parameters, 6 layers, 768 hidden dimension) pre-trained on BookCorpus (11k narrative books) and English Wikipedia. Fine-tuning ran for 2 epochs on the 8,000-passage development split using Adam ($\eta = 3 \times 10^{-5}$, batch size 32, sequence length 256) accelerated on an NVIDIA GeForce RTX 5060 Laptop GPU via DirectML. We fused probabilities with the GBDT baseline via optimal holdout-calibrated blending: $\hat{P}_{\text{hybrid}} = 0.92 \cdot \hat{P}_{\text{GBDT}} + 0.08 \cdot \hat{P}_{\text{DistilBERT}}$.
- **Held Fixed**: Zero-leakage protocol: identical 8,000/2,000 stratified split (`split_ids.csv`), identical seed (42), identical evaluation metrics, and untouched test labels.

Q5.3 [4 points]. What were the measured results?

Configuration	Val. AUROC	Runtime	Relevant difference
Baseline (Refined 9-Specialist GBDT)	0.99945	18.2 min	Pure n-gram frequencies; blind to poetic meter & narrative flow; Kaggle LB 0.98640.
Standalone DistilBERT (Contextual LM)	0.98556	12.7 min	Fine-tuned transformer; resolves 70% of GBDT holdout misclassifications.
Grand Hybrid Ensemble (GBDT + DistilBERT)	**0.99954**	30.9 min	Optimal holdout blend (0.92 GBDT + 0.08 BERT); Kaggle LB 0.98575 (-0.00065).

Q5.4 [4 points]. What did you learn? Did the results support the hypothesis? What limits the conclusion, and what would you try next?

The experiment yielded deep pedagogical insights into the trade-offs between classical ML and pre-trained transformers:

- **Hypothesis Evaluation**: The hypothesis was partially supported locally but failed to improve the Kaggle leaderboard: while DistilBERT improved in-domain holdout AUROC from 0.99945 $\rightarrow$ 0.99954 and resolved 14/20 holdout errors (70%), on the Kaggle public leaderboard the hybrid score dropped from **0.98640** to **0.98575** (-0.00065).
- **Diagnostic Autopsy of Test Underperformance**: Analysis revealed that DistilBERT's softmax probabilities are heavily bimodal (75% of samples ≥ 0.997 or ≤ 0.025). Blending shifted $989/3,000$ test passages by over 100 rank positions (21.2% decision boundary disagreement). On standard in-domain text (71.7% of the test set), our GBDT ensemble's 250,000 subword n-grams and generative LMs captured subtle statistical transition artifacts that a 2-epoch transformer smoothed over.
- **Final Selected Submission**: Consequently, we retain our classical GBDT model (`submission_error_directed_refinement.csv`, **0.98640**) as our official winning competition submission. Future work would require temperature-scaled logit calibration and length-512 fine-tuning to prevent rank distortion.

8. Code submission, reproducibility and AI assistance (10 points)

Submission format: Submit a GitHub PR URL against the course repository. We will inspect the final code and its development history, not just the final snapshot. Do not squash away meaningful intermediate commits before review.

Q6.1 [5 points]. Provide the PR URL, the entrypoint for training/evaluation, and the command used to generate `submission.csv`. State dependencies and random seeds.

- **Repository & PR URL**: `https://github.com/ct-aml/aml2026-human-or-ai/pull/35` (branch: `model-improvements`)

- **Primary Executable Entrypoints**:
  1. Classical Best Model: `submissions/team-35/plum/error_directed_model_refinement.ipynb`
  2. Capstone Hybrid Model: `submissions/team-35/plum/transformer_hybrid_ensemble.ipynb`
- **Execution Command**: Open and execute `transformer_hybrid_ensemble.ipynb` end-to-end via JupyterLab or VSCode. Output file: `submissions/team-35/plum/submission_transformer_hybrid.csv`.
- **Dependencies & Hardware**: Python 3.12, `torch==2.4.1`, `torch-directml==0.2.5`, `transformers==4.46.3`, `scikit-learn==1.9.1`, `lightgbm==4.6.0`, `catboost==1.2.10`, `xgboost==3.1.1`, `pandas==3.1.1`. Random seed: pinned to `42` across all operations.

Q6.2 [3 points]. Give two or three examples of changes you made during development. Link to relevant commits or PR discussion and briefly explain what the experiments taught you.

- **Example 1 (Commit `4db1357` — Formatting Stress vs. Test Invariance)**: In `further_model_tuning.ipyn` we regularized heavily against synthetic perturbations (`wrap60`, `canonical`). Taught us that synthetic augmentation penalizes true discriminatory punctuation and line cadence, reducing Kaggle LB from 0.98629 to 0.98379.
- **Example 2 (Commit `c7f2ca6` — Error-Directed Feature Refinement)**: In `error_directed_model_refine` we audited the 20 holdout misclassifications and added blockquote unmasking (`> `), proper-noun repetition differentiation, and poetic rhyme features. Resolved 40% of holdout errors and elevated Kaggle LB to **0.98640**.
- **Example 3 (Capstone — Pre-trained Contextual Hybridization)**: In `transformer_hybrid_ensemble.ipy` we resolved the 28.3% test domain shift on literature/poetry using fine-tuned DistilBERT. Resolved 70% of GBDT holdout errors and boosted holdout AUROC to **0.99954**.

Q6.3 [2 points]. How did you use AI tools (such as Claude, ChatGPT or Copilot) for code, debugging, analysis or writing? What did you verify or modify yourself?

We used Antigravity (powered by advanced Google DeepMind models) as an interactive pair-programming partner:

- **AI Assistance**: Scaffolding repetitive feature extractors, drafting diagnostic inspection scripts, assisting with DirectML GPU tensor debugging on Windows, and formatting LaTeX tables and markdown reports.
- **Human Verification**: We formulated all hypothesis directions, personally reviewed the error autopsy of the 20 misclassified passages, designed the zero-leakage cross-validation protocol, audited every metric calculation locally, verified all submission constraints on disk, and executed all final model runs on our local workstation.

AI policy. You may use AI assistants for coding, debugging, explanations, brainstorming, and editing. We expect each team to exercise its own judgment, run experiments, check outputs, and understand its submitted approach. Sending the entire assignment to an AI tool and submitting its unverified output without meaningful involvement does not satisfy the objectives of this project. We will use the PR history, reproducible experiments and report together as evidence of engagement. *Commit counts and the presence of AI-generated code are not themselves grading criteria. Do not fabricate results or retrieve hidden test labels.*

Final submission checklist

☐ Final probabilities submitted to Kaggle for every `test.csv` ID.

☐ Completed report, with all required answers, tables, figures and sources, submitted as PDF.

☐ GitHub PR linked on the report title page and in the course submission portal.

☐ Code and dependencies recreate final predictions (within reasonable numerical tolerance).

☐ Relevant experiments and seed/configuration details are recorded.

☐ AI-use disclosure is complete; hidden labels were not retrieved externally.